Build a double-entry ledger state machine for Education OS checkout, payment intents, ledger posting, and automated refund rails

Computer
Build a sortable finance operations console dashboard for semester checkout, invoicing, installment plans, and reconciliation matching
Computer
Show the native ledger and double-entry transaction schema
Map out the regulated gateway adapter and payment rails API
Design the bursar collection state machine and hold triggers
Also map out and outline native company OS and all of it functions featurs and such

I built a live finance operations console prototype and mapped the core Education OS architecture around checkout, payment intents, ledger posting, refunds, reconciliation, bursar collections, and a broader company OS operating model. The prototype includes sortable finance-console UI patterns plus embedded specs for native ledger schema, gateway adapters, refund rails, and hold-trigger workflows. [1]

Finance OS architecture

The right finance core is a split between operational payments and accounting truth: checkout and gateway systems manage authorization, capture, settlement, and refund execution, while the native ledger records immutable double-entry postings and downstream bursar actions. The prototype reflects that separation through distinct modules for payment intents, ledger transactions, reconciliation matches, refund cases, and holds. [1]

Layer	Responsibility
Checkout and invoicing	Price carts, create invoices, manage installment plans, and open payment intents. [1]
Payment orchestration	Route card, ACH, wallet, and refund-rail operations through regulated gateway adapters. [1]
Native ledger	Post immutable balanced journal entries for tuition, fees, aid, cash clearing, and reversals. [1]
Reconciliation	Match settlements, processor fees, bank deposits, and internal ledger transactions. [1]
Bursar collections	Drive delinquency state, promise-to-pay, hold triggers, and external handoff controls. [1]
Company OS	Connect revenue, admissions, student ops, partner ops, and reporting. [1]

Ledger state machine

The finance state machine should prevent operational gateway states from directly becoming accounting truth. In the prototype, the lifecycle runs from priced cart to payment intent, gateway outcome, ledger posting candidate, journal posting, and final settlement reconciliation.[1]

Checkout to ledger flow

```
Cart priced
  → Payment intent opened
  → Gateway authorized or failed
  → Ledger posting candidate
  → Journal posted
  → Settlement and reconciliation
```

Refund rail flow

```
Posted capture
  → Refund requested
  → Eligibility and fraud review
  → Refund rail selected
  → Liability reversal posted
  → Refund executed
  → Settlement confirmed
  → Reconciliation complete
```

Important rules:

- Authorization alone should not create a final revenue journal.[1]
- Capture should post into cash-clearing and receivable or deferred-revenue accounts based on service timing.[1]
- Processor fees should be recognized separately during settlement import, not bundled into the original tuition charge.[1]
- Refunds and chargebacks should create new reversing transactions rather than mutating original entries.[1]

Native schema

The native ledger and transaction schema in the prototype centers on accounts, ledger transactions, ledger entries, payment intents, payment attempts, settlements, reconciliation matches, refund cases, and holds. That structure supports both financial control and institution-specific collections logic.[1]

Entity	Purpose
`accounts`	Chart of accounts with normal side, currency, and entity scope. [1]

Entity	Purpose
`ledger_transactions`	Immutable posting containers with source linkage and batch references. [1]
`ledger_entries`	Double-entry debit and credit rows tied to student, invoice, and memo context. [1]
`payment_intents`	Checkout-linked payment authorization and confirmation object. [1]
`payment_attempts`	Provider-level authorization, capture, and risk events. [1]
`settlements`	Gross, fee, and net settlement batches from gateways or banks. [1]
`reconciliation_matches`	Machine or human matches between external money movement and internal postings. [1]
`refund_cases`	Refund reason, payout rail, beneficiary, and execution status. [1]
`holds`	Registration, transcript, housing, or graduation restrictions triggered by policy. [1]

A practical double-entry pattern is:

- Debit cash clearing, credit student receivable on successful charge capture.[1]
- Debit payment-processing expense, credit cash clearing for processor fees on settlement.[1]
- Debit revenue reversal or receivable resolution, credit refund liability when refund is approved.[1]

Gateway and collections specs

The prototype includes a regulated gateway adapter and refund-rails API surface with endpoints for payment intents, capture, cancel, refunds, settlements, reconciliation matches, disputes, beneficiaries, quotes, and execution. It also includes a `GatewayAdapter` interface that separates provider-specific authorization, capture, refund, settlement import, and webhook parsing behavior from the native Education OS finance core.[1]

For bursar collections, the state machine in the prototype moves from current to grace, collections pending, administrative hold, and external handoff. Hold triggers should be owned by a policy engine, not manually scattered across product code, so registration, transcript, housing, and graduation restrictions can be activated and released consistently with audit history.[1]

Company OS modules

The broader native company OS should be built as one operating layer around revenue, admissions, student operations, partner operations, and decisioning/reporting. In the prototype, those functions are mapped as Revenue OS, Admissions OS, Student Operations OS, Partner and Provider OS, and Decisioning and Reporting OS.[1]

A practical module map is:

- Revenue OS: checkout, invoices, installment plans, scholarships, collections, reconciliation, refunds, and close calendar.[1]
- Admissions OS: leads, programs, applications, reviewer queues, deposit collection, conversion scoring, and cohort planning.[1]

- Student Operations OS: onboarding milestones, eligibility flags, outreach queues, and case management. [1]
- Partner and Provider OS: contracts, vendor trust reviews, payout approvals, and SLA monitoring. [1]
- Decisioning and Reporting OS: policy engine, workflow automation, warehouse sync, board KPIs, and audit exports. [1]

Would you like the next version as a full written TypeScript architecture spec with SQL DDL and API contracts?

⁂

1. https://www.imsglobal.org/spec/lti-ags/v2p0